

Содержание

1. Цель работы	2
2. Теоретическое введение	2
2.1. Постановка задачи	2
2.2. Полиномиальная регрессия	2
2.3. Вывод системы нормальных уравнений	3
2.4. Оценка качества: MSE	3
2.5. Недообучение и переобучение	4
3. Программная реализация	4
3.1. Генерация выборки	4
3.2. Разбиение выборки	5
3.3. Полиномиальная регрессия через систему нормальных уравнений	5
4. Блок I: Генерация и визуализация выборок	6
5. Блок II: Полиномиальная регрессия	7
5.1. Три ситуации: недообучение, норма, переобучение	7
5.2. Зависимость MSE от степени полинома	8
5.2.1. Автоматическое определение оптимальной степени	8
6. Выводы	9
7. Структура проекта	9
8. Репозиторий	10

1. Цель работы

Исследование метода полиномиальной регрессии для восстановления функциональной зависимости по зашумлённым данным. Анализ явлений недообучения и переобучения, а также исследование влияния степени полинома на качество модели.

2. Теоретическое введение

2.1. Постановка задачи

Предполагается существование некоторой неизвестной функции $f : X \rightarrow Y$. Задана обучающая выборка — набор пар

$$\{(x^{(i)}, y^{(i)})\}_{i=1}^N, \quad y^{(i)} = f(x^{(i)}) + \varepsilon^{(i)},$$

где $\varepsilon^{(i)}$ — ошибка измерения, принадлежащая интервалу $[-\varepsilon_0, +\varepsilon_0]$, $\varepsilon_0 > 0$. Аргументы $x^{(i)}$ генерируются равномерно на $[-1, 1]$.

Рассматриваются два варианта распределения ошибки:

- **равномерное:** $\varepsilon \sim U[-\varepsilon_0, +\varepsilon_0]$;
- **нормальное (усечённое):** $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, где $\sigma = \varepsilon_0/3$ (правило трёх сигм), усечённое до $[-\varepsilon_0, +\varepsilon_0]$.

Рассматриваются две функции f :

- полином третьей степени: $f(x) = ax^3 + bx^2 + cx + d$, коэффициенты a, b, c, d генерируются случайно из $[-3, 3]$;
- тригонометрическая: $f(x) = x \sin(2\pi x)$.

2.2. Полиномиальная регрессия

Модель полиномиальной регрессии степени M имеет вид:

$$\hat{f}(x) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_jx^j.$$

Коэффициенты $w = (w_0, w_1, \dots, w_M)$ подбираются методом наименьших квадратов — минимизацией суммы квадратов отклонений:

$$E(w) = \sum_{k=1}^N (y_k - \hat{f}(x_k))^2 \rightarrow \min_w.$$

2.3. Вывод системы нормальных уравнений

В точке минимума все частные производные функции $E(w)$ равны нулю:

$$\frac{\partial E}{\partial w_i} = 0, \quad i = 0, 1, \dots, M.$$

Вычислим производную по w_i :

$$\frac{\partial E}{\partial w_i} = \sum_{k=1}^N 2(y_k - \hat{f}(x_k)) \cdot (-x_k^i) = 0.$$

После упрощения:

$$\sum_{k=1}^N \left(y_k - \sum_{j=0}^M w_j x_k^j \right) \cdot x_k^i = 0.$$

Раскрывая скобки и переносим слагаемые:

$$\sum_{j=0}^M w_j \underbrace{\sum_{k=1}^N x_k^{i+j}}_{a_{ij}} = \underbrace{\sum_{k=1}^N y_k x_k^i}_{b_i}.$$

Таким образом, получаем систему из $M + 1$ линейных уравнений — **систему нормальных уравнений**:

$$Aw = b,$$

где матрица A и вектор b определяются формулами:

$$a_{ij} = \sum_{k=1}^N x_k^{i+j}, \quad b_i = \sum_{k=1}^N y_k \cdot x_k^i, \quad i, j = 0, 1, \dots, M.$$

Матрица A симметрична ($a_{ij} = a_{ji}$) и положительно определена при достаточном числе различных точек x_k , что гарантирует единственность решения.

2.4. Оценка качества: MSE

Качество модели оценивается среднеквадратичной ошибкой:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \left(y^{(i)} - \hat{f}(x^{(i)}) \right)^2.$$

Различают MSE на обучающей выборке (train) и на тестовой (test).

2.5. Недообучение и переобучение

Недообучение

Степень M слишком мала. Модель не может описать форму данных.

MSEtrain высокий

MSEtest высокий

Норма

Оптимальная степень M . Модель хорошо обобщается.

MSEtrain низкий

MSEtest низкий

Переобучение

Степень M слишком велика. Модель запоминает шум.

MSEtrain ≈ 0

MSEtest большой

3. Программная реализация

3.1. Генерация выборки

Параметры по умолчанию:

- Seed для генератора случайных чисел: SEED = 42
- Размер выборки для блока I: $N = 100$ (можно изменить через переменную окружения)
- Для блока II: $N = 15$ (по умолчанию, также настраивается)
- Значения ε_0 варьируются в зависимости от эксперимента: 0.2, 0.3, 0.5

Исходный файл: `functions.py`

```
import numpy as np
from scipy.stats import truncnorm
from config import a, b, c, d # Коэффициенты из config.py

def f_poly(x):
    return a * x**3 + b * x**2 + c * x + d

def f_sin(x):
    return x * np.sin(2 * np.pi * x)

def generate_error_uniform(n, eps0):
    return np.random.uniform(-eps0, eps0, size=n)

def generate_error_normal(n, eps0):
    """Нормальная ошибка, усечённая до [-eps0, eps0].
    Правило трёх сигм: sigma = eps0 / 3."""
    sigma = eps0 / 3
    a_trunc = -eps0 / sigma # = -3
    b_trunc = eps0 / sigma # = +3
    return truncnorm.rvs(a_trunc, b_trunc, scale=sigma, size=n)

def generate_sample(f, n, eps0, error_type='uniform'):
    x = np.random.uniform(-1, 1, size=n)
```

```

if error_type == 'uniform':
    eps = generate_error_uniform(n, eps0)
else: # 'normal'
    eps = generate_error_normal(n, eps0)

y = f(x) + eps
return x, y

```

3.2. Разбиение выборки

Выборка разбивается на обучающую (80%) и тестовую (20%) части. Перед разбиением индексы перемешиваются случайно.

Исходный файл: regression.py

```

def train_test_split(x, y, test_ratio=0.2):
    n = len(x)
    indices = np.random.permutation(n) # случайный порядок
    split = int(n * (1 - test_ratio)) # граница разбиения

    train_idx = indices[:split]
    test_idx = indices[split:]

    return x[train_idx], y[train_idx], x[test_idx], y[test_idx]

```

3.3. Полиномиальная регрессия через систему нормальных уравнений

Реализация непосредственно по формулам из теоретической части.

Исходный файл: regression.py

```

def fit_polynomial(x_train, y_train, degree):
    M = degree
    # Матрица A: A[i,j] = сумма x_k^(i+j)
    A = np.zeros((M + 1, M + 1))
    for i in range(M + 1):
        for j in range(M + 1):
            A[i, j] = np.sum(x_train ** (i + j))

    # Вектор b: b[i] = сумма y_k * x_k^i
    b = np.zeros(M + 1)
    for i in range(M + 1):
        b[i] = np.sum(y_train * (x_train ** i))

    # Решаем систему A * w = b
    w = np.linalg.solve(A, b)
    return w # w[0]=свободный член, w[1]=коэф при x, w[2]=коэф при x^2...

```

```
def predict(x, w):
    """y = w[0] + w[1]*x + w[2]*x^2 + ..."""
    y_pred = np.zeros_like(x, dtype=float)
    for i, wi in enumerate(w):
        y_pred += wi * (x ** i) # w[0]*x^0 + w[1]*x^1 + w[2]*x^2 + ...
    return y_pred

def mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)
```

4. Блок I: Генерация и визуализация выборок

Было сгенерировано 6 вариантов выборок с параметрами:

- Размер выборки: $N = 100$ (по умолчанию, настраивается через GUI)
- Для обеих функций ($f = ax^3 + bx^2 + cx + d$ и $f = x \sin(2\pi x)$) использованы значения:
 - $\varepsilon_0 = 0.2$ с равномерной ошибкой
 - $\varepsilon_0 = 0.5$ с равномерной ошибкой
 - $\varepsilon_0 = 0.5$ с нормальной (усечённой) ошибкой

Исходный файл: `block1_main.py`

На рисунке 1 показаны результаты: синяя линия — истинная функция $f(x)$, красные (tomato) точки — выборка с шумом, голубая полоса — коридор $\pm\varepsilon_0$.

Задание 1. Блок I — Генерация выборок

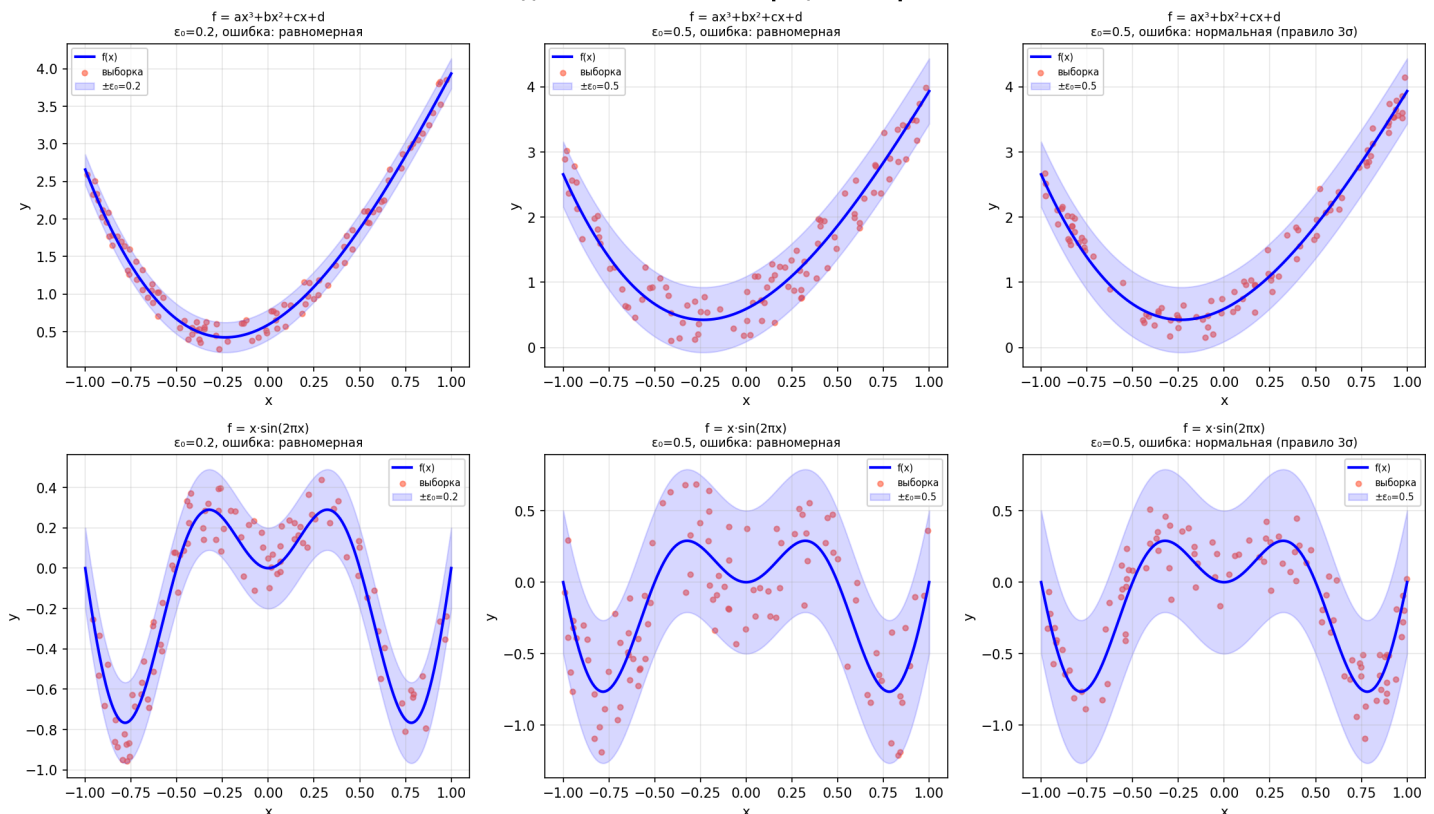


Рис. 1. Сгенерированные выборки для разных ε_0 и типов ошибки. Верхний ряд — полином $f = ax^3 + bx^2 + cx + d$, нижний ряд — $f = x \sin(2\pi x)$.

5. Блок II: Полиномиальная регрессия

5.1. Три ситуации: недообучение, норма, переобучение

Для блока II используются следующие параметры:

- Размер выборки: $N = 15$ (по умолчанию, меньше для демонстрации переобучения)
- Значение шума: $\varepsilon_0 = 0.3$
- Тип ошибки: нормальная (усечённая)
- Разбиение: 80% обучающая, 20% тестовая выборка

На рисунке 2 представлены результаты полиномиальной регрессии при трёх различных степенях полинома.

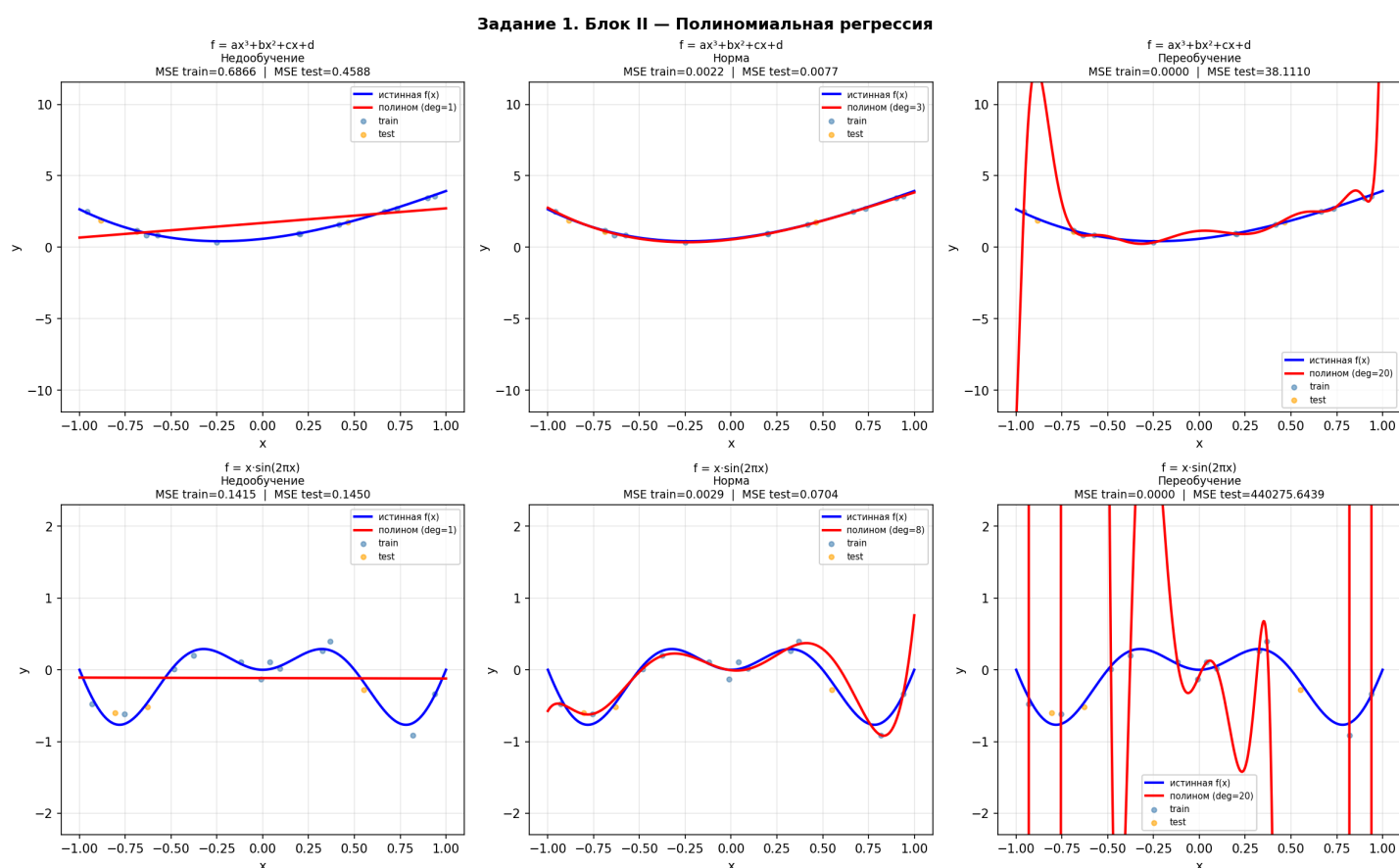


Рис. 2. Полиномиальная регрессия. Синяя линия — истинная функция, красная — подобранный полином, сине-стальные точки — обучающая выборка, оранжевые — тестовая.

В таблице ниже приведены значения MSE для функции $f = ax^3 + bx^2 + cx + d$:

Степень M	Ситуация	MSE train	MSE test
1	Недообучение	0.6866	0.4588
3	Норма	0.0022	0.0077
20	Переобучение	0.0000	38.1110

Таблица 1. Значения MSE для полинома $f = ax^3 + bx^2 + cx + d$. Конкретные значения зависят от случайной генерации коэффициентов.

Наблюдение: при степени $M = 20$ MSE на тестовой выборке значительно выше, чем при оптимальной степени $M = 3$, — классический признак переобучения.

5.2. Зависимость MSE от степени полинома

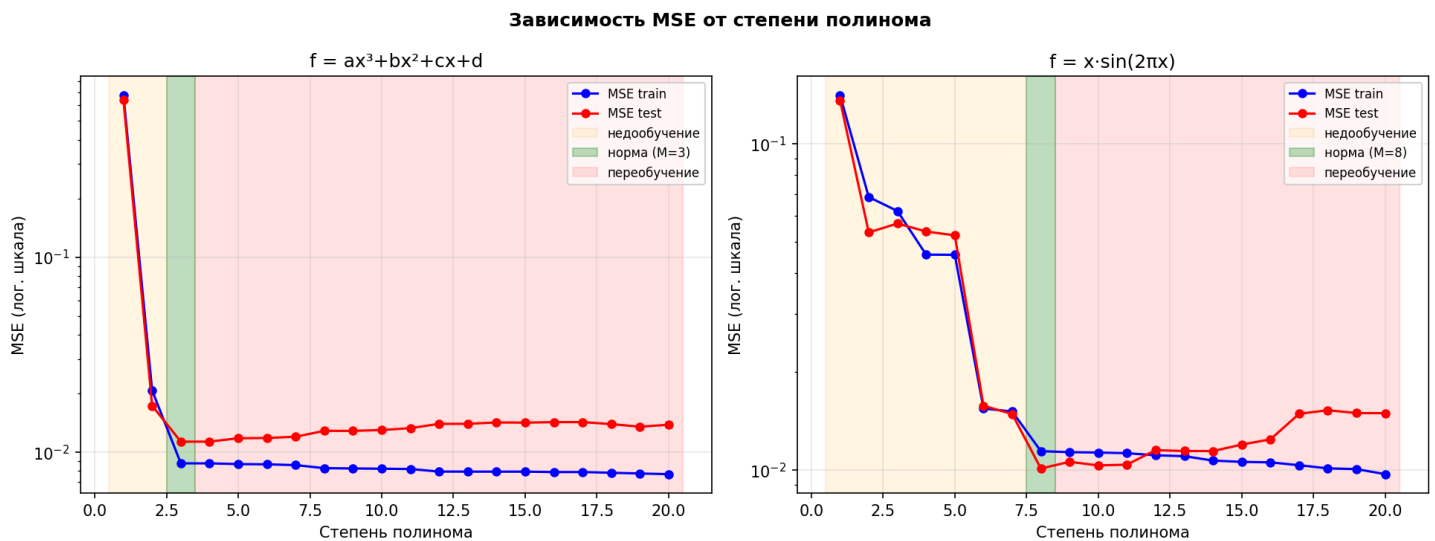


Рис. 3. Зависимость MSE от степени полинома M в логарифмической шкале. Синяя линия — MSE на обучающей выборке, красная — на тестовой.

На графике отчётливо видна U-образная кривая тестовой ошибки:

- **левая ветвь** ($M < \text{оптимального}$) — недообучение: обе ошибки высокие;
- **дно U-кривой** — оптимальная сложность модели: обе ошибки низкие;
- **правая ветвь** ($M > \text{оптимального}$) — переобучение: train-ошибка продолжает падать, test-ошибка растёт.

5.2.1. Автоматическое определение оптимальной степени

В реализации добавлен алгоритм автоматического поиска оптимальной степени полинома:

Исходный файл: `block2_main.py` (функция `plot_mse_vs_degree`)

```
best_idx = int(np.argmin(mse_tests))
best_degree = degrees[best_idx]
```


Оптимальная степень определяется как та, при которой MSE на тестовой выборке минимален. На графике зоны недообучения, нормы и переобучения выделяются динамически относительно найденной оптимальной степени:

- **Недообучение** (оранжевая зона): от $M = 1$ до $M = \text{best_degree} - 1$
- **Норма** (зелёная зона): $M = \text{best_degree}$ (оптимальная степень)
- **Переобучение** (красная зона): от $M = \text{best_degree} + 1$ до $M = 20$

Это позволяет наглядно видеть границы между режимами работы модели для каждой конкретной функции и размера выборки.

6. Выводы

В ходе работы были достигнуты следующие результаты:

1. Реализована генерация зашумлённых выборок для двух типов функций и двух типов распределения ошибки измерения (равномерного и нормального усечённого по правилу трёх сигм).
2. Реализован метод полиномиальной регрессии через решение системы нормальных уравнений $Aw = b$ с помощью `np.linalg.solve`.
3. Проведён анализ явлений недообучения и переобучения:
 - малые степени ($M = 1$) — недообучение, обе ошибки высокие;
 - оптимальные степени ($M \approx 3-5$) — хорошее обобщение;
 - большие степени ($M \geq 15$) — переобучение, тестовая ошибка резко возрастает.
4. Оптимальная степень полинома определяется по минимуму MSE на тестовой выборке. Характерная картина — U-образная кривая тестовой ошибки на графике зависимости MSE от M .
5. Нормальный шум даёт более стабильные результаты по сравнению с равномерным, так как большинство значений ошибки сосредоточено около нуля.

7. Структура проекта

Проект организован по модульному принципу для удобства разработки и поддержки:

```
SMGMO/  
├─ config.py           # Конфигурация и параметры  
├─ functions.py        # Математические функции (f_poly, f_sin)  
│                       # и генерация выборок  
├─ regression.py       # Функции для полиномиальной регрессии  
│                       # (train_test_split, fit_polynomial, predict, mse)  
└─ block1_main.py      # Блок I: генерация и визуализация выборок
```

```
|— block2_main.py      # Блок II: полиномиальная регрессия
|— gui_launcher.py    # Графический интерфейс для запуска блоков
|— task1_block1.png   # Выходное изображение блока I
|— task1_block2.png   # Выходное изображение блока II
|— task1_block2_mse.png # График зависимости MSE от степени
└— main.typ           # Исходник отчёта (Typst)
```

8. Репозиторий

Исходный код проекта доступен на GitHub: <https://github.com/Q1zin/SMGMO.git>